MEGN 651A: Homework Two

Rachel Bertaud

Feburary 10th, 2026

The code used to produce this work can be located at this address:

https://github.com/rachbertaud/Advanced_CFD/tree/main/02_variable_conductivity_heat_conduction.

For reproduction of results, download the folder `02_variable_conductivity_heat_conduction`, call `$ make runAll`, call `$ make ErrorPlots` and then run the corresponding MATLAB files to produce pictures and results:

- `err_no_k.m`: produces and saves error plot for No K case.
- `vis_no_k.m`: Produces visualization of Numerical and Analytical Solution for the No K case.
- `err_w_k.m.m`: Produces and saves error plot for With Km case.
- `vis_k_m.m`: Produces visualization of Numerical and Analytical Solution for the With Km case.
- `fuel_rod_plot.m`: Produces visualization of Numerical Solution for the Fuel Rod Problem.

`$ make clean` removes all the executables and `$ make clean_data` removes all the `txt` files.

Overview of Code

The purpose of this code is to numerically solve the steady, two-dimensional diffusion equation at discrete locations using a finite-volume method. Our goal is to simulate the steady temperature distribution in a nuclear fuel rod immersed in a liquid coolant.

Before we get there, we must consider the following governing equation:

$$\int_{CS} \mathcal{D}(\mathbf{x})(\nabla c \cdot \mathbf{n})\ dA = -\int_{CV} f(\mathbf{x})\ dV.$$

This can be discretized, given discrete points of $f(x, y)$, as

$$a_P c_P = a_E c_E + a_N c_N + a_W c_W + a_S c_S + b,$$

where

$$a_E = \frac{\mathcal{D}_e \Delta y}{x_E - x_P}, \qquad a_W = \frac{\mathcal{D}_w \Delta y}{x_P - x_W}$$

$$a_N = \frac{\mathcal{D}_n \Delta x}{y_N - y_P}, \qquad a_S = \frac{\mathcal{D}_s \Delta x}{y_P - y_S}$$

$$a_P = a_E + a_N + a_W + a_S, \qquad b = f_P\ \Delta x\ \Delta y.$$

Note that we will analytically solve for $c(x, y)$ using linear algebra. We solve

$$\mathbf{Ac} = \mathbf{x}$$

for $\mathbf{c}$ given $\mathbf{x}$ and $\mathbf{A}$, which are built using the discretized equations above. We will assume Dirichlet boundary conditions throughout all steps.

Verification

Our first goal is to verify the designed code works as anticipated. We first define our manufactured solution and $f(x, y)$ for the verification steps as

$$c_m(x, y) = \cos(x)\cos(y),$$
$$f(x, y) = 2\cos(x)\cos(y).$$

We discretize as defined in the overview, with $\mathcal{D}(x, y) = 1$ and our boundary conditions defined as our manufactured solution at their respective locations. We then build $\mathbf{A}$ and $\mathbf{b}$, and solve for $\mathbf{c}$. Using the results of this step, we verify that

1. Our solution's relative error measured with the infinity norm converges with order two accuracy and
2. Our operator, $\mathbf{A}$, computes the expected solution.

Our next goal is to verify that defining a variable $\mathcal{D}(x, y)$ term does not change our solution. This is done by introducing

$$k_m(x, y) = 1 + 0.5\sin(x)\sin(y).$$

We can then re-write our steady-state diffusion PDE as

$$\nabla \cdot [k_m(x, y)\nabla c_m(x, y)] = -f(x, y)$$

which produces

$$f(x, y) = 2\cos(x)\cos(y)(1 + \sin(x)\sin(y)).$$

We repeat the same process of building $\mathbf{A}$ and $\mathbf{b}$, and then solving for $\mathbf{c}$. Note that the boundary conditions do not change from the last step. We follow the same steps to verify our solution as defined previous.

The results of the verification process are defined below in the "Results" section.

Nuclear Fuel Rod Application

Upon verification, we study the following problem. Refer to Figure 1 for a stencil of the fuel cell. Consider a domain defined as $x \in [0, L_x]$ and $y \in [0, L_y]$. The axis of the rod is located at $(x_c, y_c) = (0.5L_x, 0.5L_y)$. The rod consists of an inner fuel core of radius R_1 and thermal conductivity k_1, surrounded by a cladding of outer radius R_2 and thermal conductivity k_2. The rest of the domain is considered coolant with thermal conductivity k_3. We will define the radial distance from the rod center as $r(x, y)$ and a volumetric heat source $s(x, y)$ that varies with $r(x, y)$ as follows:

$$r(x, y) = \sqrt{(x - x_c)^2 + (y - y_c)^2}$$
$$s(x, y) = a\left[1 + b\left(\frac{r}{R_1}^2\right)\right],$$

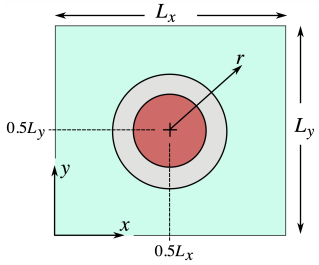


Figure 1: Stencil of fuel cell.

a, b are user-defined constants. We assume no heat generation outside of the core, the fluid is at rest, and no radiative heat transfer. We model cooling by applying a constant temperature T_c at the boundaries, defined by the user. For simulations of the fuel rod, we assume

$$N_x = 100, \quad N_y = 100, \quad L_x = 0.015 \text{ m}, \quad L_y = 0.015 \text{ m},$$
$$R_1 = 0.004 \text{ m}, \quad R_2 = 0.005 \text{ m}, \quad k_1 = 2.0 \text{ W/m/K}, \quad k_2 = 15.0 \text{ W/m/K},$$
$$k_3 = 0.6 \text{ W/m/K}, \quad a = 1.0 \times 10^7 \text{ W/m}^3, \quad b = 1.0, \quad T_c = 300.0 \text{ K}.$$

We solve for the function of temperature distribution, $T(x, y)$, of the fluid caused by this source term. The results are discussed in the "Results" section.

Error Analysis

During the verification process, error of the numerical solution is calculated as relative error using the infinity norm, which is defined as

$$E_\infty = \frac{||c_{\text{numerical}} - c_{\text{analytical}}||_\infty}{||c_{\text{analytical}}||_\infty} = \frac{\max|c_{\text{numerical}} - c_{\text{analytical}}|}{\max|c_{\text{analytical}}|}.$$

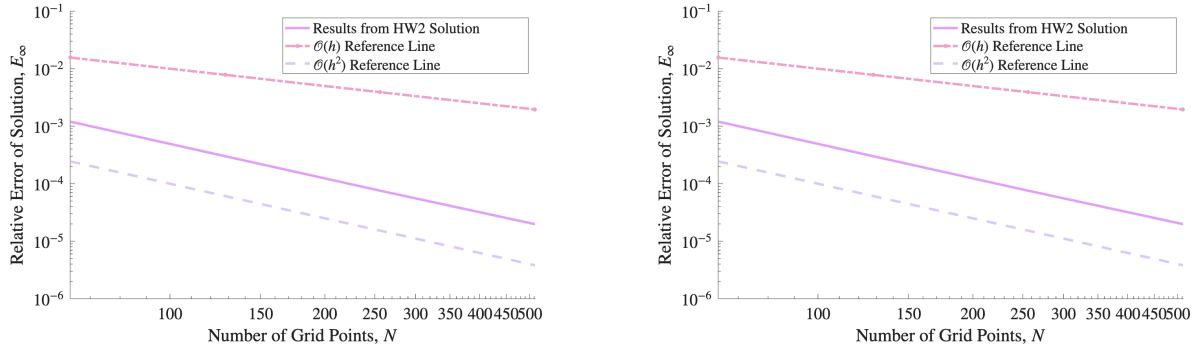
Note that we will also define local error as

$$\text{Local Error} = |c_{\text{numerical}} - c_{\text{analytical}}|.$$

Results

Overall, the results of this code verified its capabilities and produced a intuitive solution to the fuel cell application problem. Everything performed as anticipated. Refer to

- Figure 2 for the error plots for the different verification cases.
- Figure 3 for the results of the verification with constant diffusivity.
- Figure 4 for the results of the verification with variable diffusivity.
- Figure 5 for the results of the fuel cell application run. Note that the boundaries in subfigure (a) are indeed 300K, which we prescribed them to be.

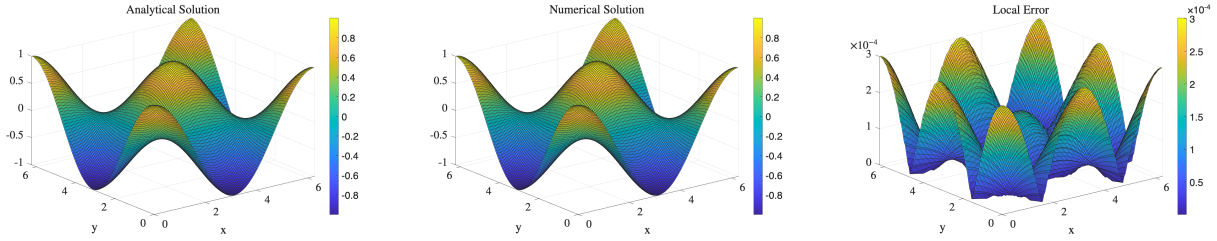


(a) Constant diffusivity case

(b) Variable diffusivity case

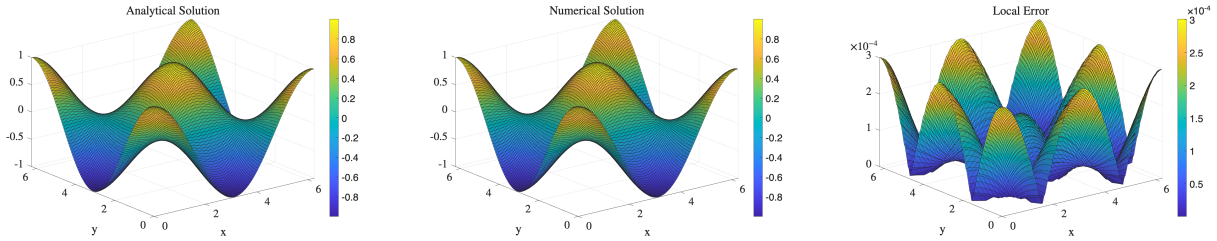
Figure 2: Relative error versus number of grid points for verification cases. Both cases converge with second-order accuracy, which is what we anticipate.

Below are the visual solutions of our simulation alongside the analytical solution for $L_x = L_y = 2\pi$ & $N_x = N_y = 128$. We can be confident in the accuracy of the code from the local error plot, noting higher errors near the boundaries.



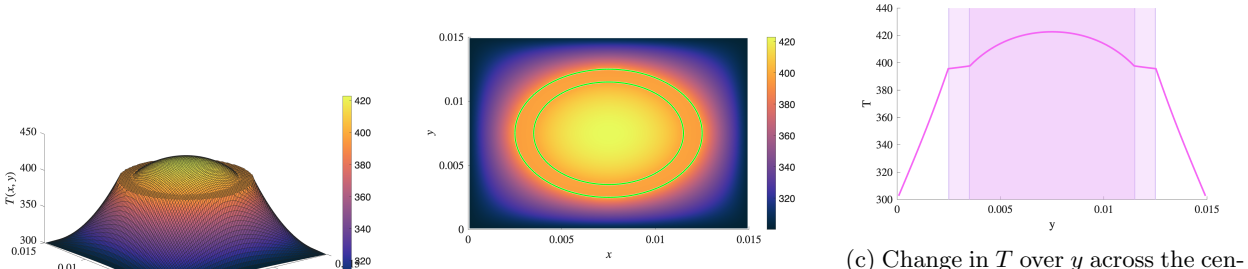
(a) Analytical Solution (b) Numerical Solution (c) Local Error

Figure 3: Results of the simulation code with constant diffusivity.



(a) Analytical Solution (b) Numerical Solution (c) Local Error

Figure 4: Results of the simulation code with variable diffusivity.



(a) Analytical Solution of $T(x, y)$.

(b) 2D colormap of solution with superimposed circles representing the cladding of the fuel cell.

(c) Change in T over y across the center line x_c, with variable shading depicting the center and cladding of the fuel cell.

Figure 5: Results of the simulation code with variable diffusivity.

Code

For the sake of page usage, I am only including the Fuel Rod code and the functions code. Everything else that was used to produce these results can be found at this GitHub location. Please refer to the first page for reproduction instructions.

HW2_Fuel_Rod.cpp

```cpp
#include <fstream>      // needed for file writing
#include <iostream>     // needed for cout
#include <string>       // needed for string
#include <map>          // for variable map definition
#include "../HW2.h"     // for user-defined functions
#include <vector>       // needed for vectors
#include <cmath>        // math library for RHS
#include <Eigen/Sparse>
#include <Eigen/Dense>
#include <Eigen/SparseLU>

int main(int argc, char *argv[])
{
        std::map<std::string, double> var;
        int input_flag = atoi(argv[1]);

        read_inputs(var, input_flag);



        int Nx = int(var["Nx"]);
        int Ny = int(var["Ny"]);
        double Lx = var["Lx"];
        double Ly = var["Ly"];
        double R1 = var["R1"];
        double R2 = var["R2"];
        double k1 = var["k1"];
        double k2 = var["k2"];
        double k3 = var["k3"];
        double a = var["a"];
        double b = var["b"];
        double Tc = var["Tc"];

        std::cout << "--------INPUTS---------" << std::endl;
        for(const auto& pair : var)
        {
                std::cout << pair.first << " = " << pair.second << std::endl;
        }

        // defining dx and dy with user inputs
        double dx = Lx / static_cast<double>(Nx);
        double dy = Ly / static_cast<double>(Ny);

        // defining x-domain vector spaces
        std::vector<double> xf(var["Nx"] + 1); // face locations
        std::vector<double> x(var["Nx"] + 2); // cell-centered locations

        // computing face locations
        for(int i = 0; i <= Nx; i++)
        {
```

```
51                  xf[i] = dx*i;
52          }
53
54          // computing cell centered loations
55          for(int i = 0; i <= (Nx + 1); i++)
56          {
57                  x[i] = -dx*0.5 + dx*i;
58          }
59
60          // defining y-domain vector spaces
61          std::vector<double> yf(Ny + 1); // face locations
62          std::vector<double> y(Ny + 2); // cell-centered locations
63
64          // computing face locations
65          for(int i = 0; i <= Ny; i++)
66          {
67                  yf[i] = dy*i;
68          }
69
70          // computing cell centered loations
71          for(int i = 0; i <= (Ny + 1); i++)
72          {
73                  y[i] = -dy*0.5 + dy*i;
74          }
75
76          std::ofstream xfile;
77          std::ofstream yfile;
78          xfile.open("Fuel_Rod/x_Fuel_Rod.dat");
79          yfile.open("Fuel_Rod/y_Fuel_Rod.dat");
80
81          for (int i = 0; i < (Nx + 2); i++)
82          {
83                  xfile << x[i] << "\n";
84          }
85
86          for (int i = 0; i < (Ny + 2); i ++)
87          {
88                  yfile << y[i] << "\n";
89          }
90
91          xfile.close();
92          yfile.close();
93
94  //--------------------------------------- PROBLEM ONE
        ---------------------------------------
95
96          // build matrix
97          int row;
98          int Nx2 = Nx + 2;
99          int Ny2 = Ny + 2;
100
101         std::vector<int> ROW, COL;
102         std::vector<double> VAL;
103
104         // north/south boundary conditions
105         for( int i = 1; i <= Nx; i ++)
106         {
107                 //southern (first row)
108                 row = i;
```

```
109
110                 //these enforce that the average between GC and domain is 0
111                 ROW.push_back(row);
112                 COL.push_back(row);
113                 VAL.push_back(0.5);
114
115                 ROW.push_back(row);
116                 COL.push_back(row + Nx2);
117                 VAL.push_back(0.5);
118
119                 //northern (last row)
120                 row = ((Ny2 - 1)*Nx2) + i;
121
122                 ROW.push_back(row);
123                 COL.push_back(row);
124                 VAL.push_back(0.5);
125
126                 ROW.push_back(row);
127                 COL.push_back(row - Nx2);
128                 VAL.push_back(0.5);
129         }
130
131         // east/west BC
132         for( int j = 1; j <= Ny; j++)
133         {
134                 // western (left-most column)
135                 row = j*Nx2;
136
137                 ROW.push_back(row);
138                 COL.push_back(row);
139                 VAL.push_back(0.5);
140
141                 ROW.push_back(row);
142                 COL.push_back(row + 1);
143                 VAL.push_back(0.5);
144
145                 // eastern (right-most column)
146                 row = (j + 1)*Nx2 - 1; // remember we don't care about the corners
147
148                 ROW.push_back(row);
149                 COL.push_back(row);
150                 VAL.push_back(0.5);
151
152                 ROW.push_back(row);
153                 COL.push_back(row - 1);
154                 VAL.push_back(0.5);
155         }
156
157         // five-point stencil
158         double ae = dy/dx;
159         double aw = dy/dx;
160         double an = dx/dy;
161         double as = dx/dy;
162         double ap = ae + aw + an + as;
163
164         double kp;
165         double ke;
166         double kw;
167         double kn;
```

```
168         double ks;
169
170
171
172         for ( int i = 1; i <= Nx; i++ )
173         {
174                 for ( int j = 1; j <= Ny; j++ )
175                         {
176                         row = Nx2*j + i;
177
178                         ke = fuel_therm_cond(xf[i], y[j], Lx, Ly, R1, R2, k1, k2,
                                k3);
179                         kw = fuel_therm_cond(xf[i - 1], y[j], Lx, Ly, R1, R2, k1,
                                k2, k3);
180                         kn = fuel_therm_cond(x[i], yf[j], Lx, Ly, R1, R2, k1, k2,
                                k3);
181                         ks = fuel_therm_cond(x[i], yf[j - 1], Lx, Ly, R1, R2, k1,
                                k2, k3);
182
183                         ae = ke*dy/dx;
184                         aw = kw*dy/dx;
185                         an = kn*dx/dy;
186                         as = ks*dx/dy;
187                         ap = ae + aw + an + as;
188
189
190                         //these will be all the points not on the boundaries.
191                         ROW.push_back(row);
192                         COL.push_back(row);
193                         VAL.push_back(ap);
194
195                         ROW.push_back(row);
196                         COL.push_back(row + 1);
197                         VAL.push_back(-ae);
198
199
200                         ROW.push_back(row);
201                         COL.push_back(row - 1);
202                         VAL.push_back(-aw);
203
204                         ROW.push_back(row);
205                         COL.push_back(row + Nx2);
206                         VAL.push_back(-an);
207
208                         ROW.push_back(row);
209                         COL.push_back(row - Nx2);
210                         VAL.push_back(-as);
211
212                 }
213
214         }
215
216         // now for extraneous nodes
217
218         // bottom left
219         ROW.push_back(0);
220         COL.push_back(0);
221         VAL.push_back(1);
222
```

```
223         //bottom right
224         ROW.push_back(Nx2 - 1);
225         COL.push_back(Nx2 - 1);
226         VAL.push_back(1);
227 
228         //top left
229         ROW.push_back((Ny+1)*Nx2);
230         COL.push_back((Ny + 1)*Nx2);
231         VAL.push_back(1);
232 
233         // top right
234         ROW.push_back(Nx2*Ny2 - 1);
235         COL.push_back(Nx2*Ny2 - 1);
236         VAL.push_back(1);
237 
238 //---------------------------PROBLEM TWO---------------------------------
239         // initalize RHS vector to zeros
240         int Ntot = Ny2*Nx2; // total num of nodes
241         std::vector<double> B(Ntot, 0.0);; // init to 0
242 
243         // south/north boundary conditions
244         for(int i = 1; i <= Nx; i++)
245         {
246                 // south
247                 row = i;
248                 B[row] = Tc;
249 
250                 // north
251                 row = (Ny + 1)*Nx2 + i;
252                 B[row] = Tc;
253 
254         }
255 
256         // east/west boundary conditions
257         for(int j = 1; j <= Ny; j++)
258         {
259                 // west
260                 row = Nx2*j;
261                 B[row] = Tc;
262 
263                 // east
264                 row = Nx2*(j + 1) - 1; //don't forget the first row corner is meh
265                 B[row] = Tc;
266 
267         }
268 
269 
270         // internal nodes
271         for (int i = 1; i <= Nx; i++)
272         {
273                 for (int j = 1; j <= Ny; j++)
274                 {
275                         row = Nx2*j + i;
276                         double r = find_r(x[i],y[j],Lx,Ly);
277                         double source_term = source(r, a, b, R1);
278 
279                         B[row] =dx*dy*source_term;
280                 }
281         }
```

```
282
283
284 //-------------------------PROBLEM THREE----------------------------------
285
286         Eigen::VectorXd RHS(Ntot);
287         for (int i = 0; i < Ntot; i++)
288         {
289                 RHS[i] = B[i];
290         }
291
292         // To define A
293         int nnz = ROW.size();
294
295         std::vector<Eigen::Triplet<double>> triplets;
296         triplets.reserve(nnz);
297
298         for (int k = 0; k < nnz; k++)
299         {
300                 triplets.push_back(Eigen::Triplet<double>(
301                         ROW[k],
302                         COL[k],
303                         VAL[k]
304                 ));
305         }
306
307         Eigen::SparseMatrix<double> A(Ny2*Nx2, Nx2*Ny2);
308         A.setFromTriplets(triplets.begin(), triplets.end());
309
310
311         // Create Solver
312         Eigen::SparseLU<Eigen::SparseMatrix<double>> solver;
313
314         // Factorize matrix
315         solver.compute(A);
316         if(solver.info() != Eigen::Success)
317         {
318                 std::cerr << "Decomposition failed! \n";
319                 return -1;
320         }
321
322         // Solve the system
323         Eigen::VectorXd C = solver.solve(RHS);
324         if(solver.info() != Eigen::Success)
325         {
326                 std::cerr << "Solving failed! \n";
327                 return -1;
328         }
329
330         // Save the solution to file
331         std::ofstream cfile("Fuel_Rod/c_Fuel_Rod.dat");
332         for (int i = 0; i < Ntot; i++)
333         {
334                 cfile << C[i] << "\n";
335         }
336         cfile.close();
337
338 //this ends main
339 }
```

funcs_HW2.cpp

```cpp
#include <fstream>      // helps us read files
#include <iostream>     //needed for cout
#include <sstream>      // needed for stringstream
#include <string>       //needed for string
#include <map>          // for variable map definition
#include "HW2.h"

void read_inputs(std::map<std::string, double>& variables, int input_flag)
{
        //define a string
        std::string line;
        std::string var_name;

        //open the input file
        std::ifstream inputfile;
        if(input_flag == 1)
        {
                inputfile.open("inputs_error");
        }
        else if(input_flag == 2)
        {
                inputfile.open("inputs_base");
        }
        else
        {
                inputfile.open("inputs");
        }
        double value;
        int i = 0;

        while(std::getline(inputfile, line))
        {
                if(line=="endfile") break;

                if(i % 2 == 0){
                        std::stringstream(line) >> var_name;
                }
                else{
                        std::stringstream(line) >> value;
                        variables[var_name] = value;
                }

                i++;
        }

        inputfile.close();
}

double var_therm_cond(double x, double y){

        double km;
        km = 1 + 0.5*std::sin(x)*std::sin(y);

        return km;
}

```

```
57
58  double find_r(double x, double y, double Lx, double Ly){
59
60          double r;
61
62          double x_diff = std::pow((x - (Lx/2)), 2);
63          double y_diff = std::pow((y - (Ly/2)),2);
64
65          r = std::sqrt(x_diff + y_diff);
66          //std::cout << "r = " << r << "(x, y) : " << x << ", " << y << std::endl;

67          return r;
68  }
69
70  double fuel_therm_cond(double x, double y, double Lx, double Ly, double R1, double
        R2, double k1, double k2, double k3){
71
72
73          double r = find_r(x, y, Lx, Ly);
74
75          double k = k3;
76
77          if ( r < R1 )
78          {
79                  k = k1;
80          }
81          else if ( r < R2 )
82          {
83                  k = k2;
84          }
85
86          return k;
87
88  }
89
90  double source(double r, double a, double b, double R1){
91
92          double source_val = a*(1 + b*(std::pow((r/R1),2)));
93
94          if ( r > R1 ) {
95                  source_val = 0;
96          }
97
98          return source_val;
99  }
```